Shared-memory parallelism with OpenMP

Correct work sharing, one example at a time

Chapter 7
Instructor teaching deck

Three questions guide every example

01 Who writes each result?

02 When may another thread read it?

03 Which OpenMP construct expresses that rule?

Build understanding in three stages

01 Create a team and distribute independent loops.

02 Combine results and choose data scope.

03 Reuse teams, then express irregular work as tasks.

REFERENCE

Choose an OpenMP-capable compiler

Platform	Install / environment	Compiler
macOS	Homebrew: brew install gcc	gcc-16*
Ubuntu / Debian	sudo apt install gcc	gcc
Windows	MSYS2 UCRT64 + GCC + libgomp	gcc
Windows alternative	Ubuntu inside WSL	gcc

*Use the installed Homebrew GCC version. Commands are run from the project root.

Verify the compiler and runtime

```
gcc-16 -std=c11 -O2 -Wall -Wextra \
  -Werror -fopenmp \
  examples/chapter7/check_openmp.c \
  -o check_openmp

OMP_NUM_THREADS=4 ./check_openmp

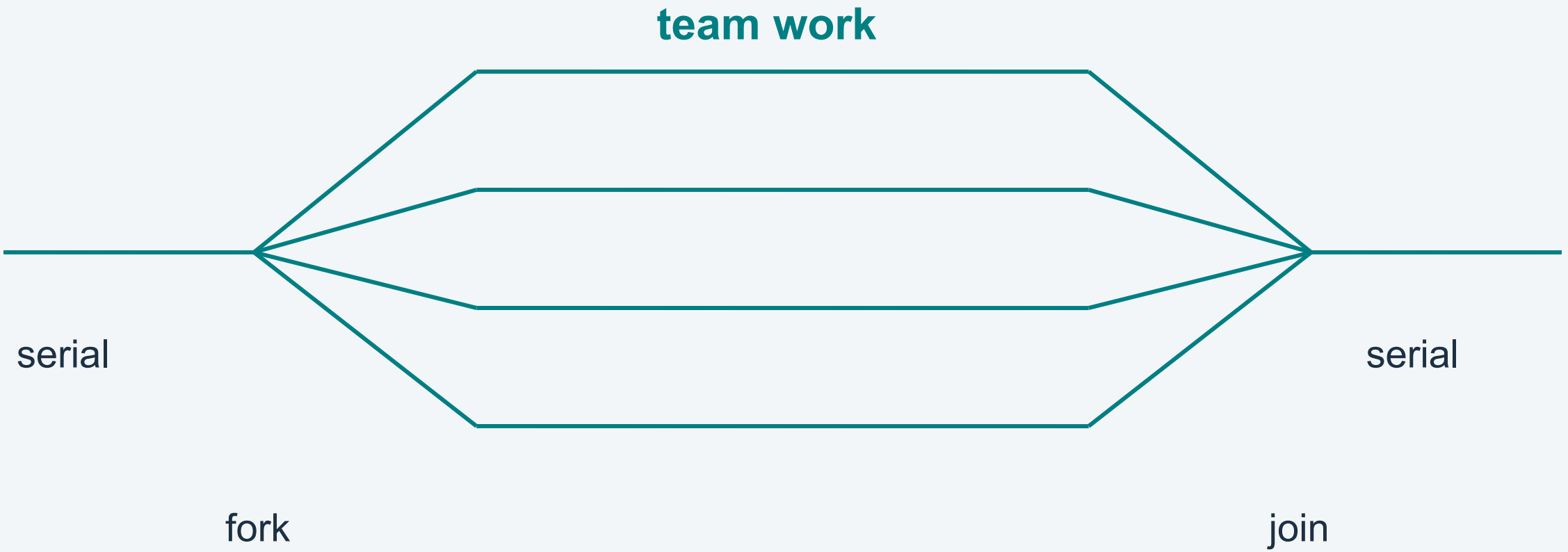
# Typical result:
team=4 counted=4
```

The flag enables OpenMP.

The two counts must agree.

Thread count is a request.

A team executes between fork and join



REFERENCE

Team creation and work sharing differ

Construct	What it does
parallel	Forms a team; each thread enters the block
for	Distributes iterations within an existing team
parallel for	Combines team creation and loop distribution

Scale an array with independent writes

```
#pragma omp parallel for \
    default(none) shared(a,b,n) \
    schedule(static)
for (int i = 0; i < n; ++i)
    b[i] = 2 * a[i];
```

Each i writes only b[i].

The input a is read-only.

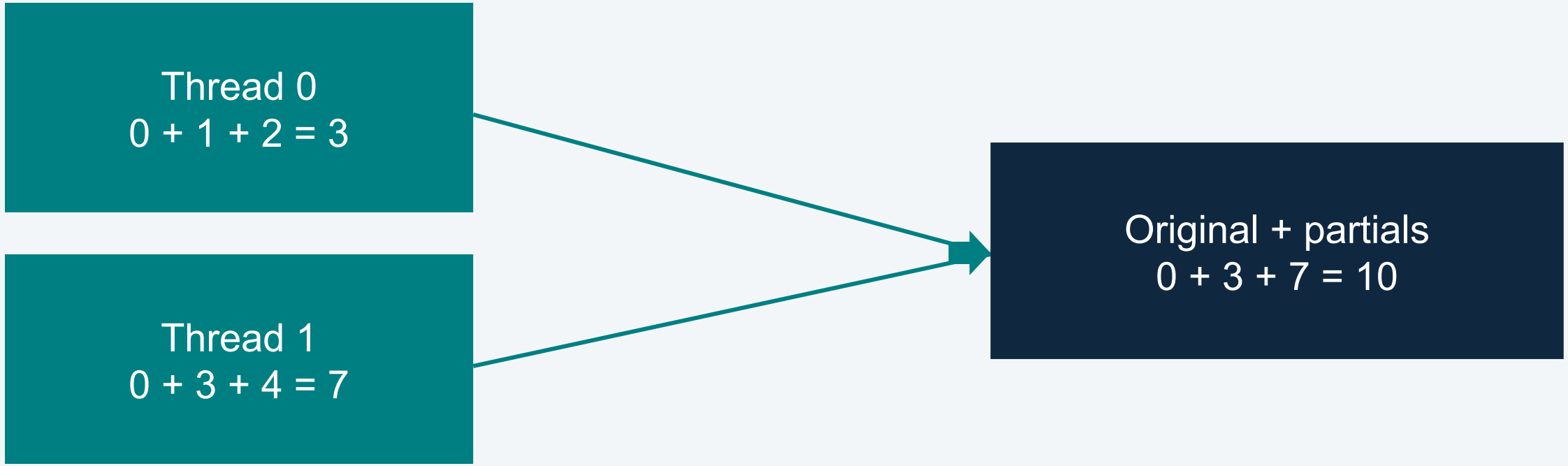
No two iterations own the same output.

A shared accumulator needs a reduction

```
long long total = 0;
#pragma omp parallel for \
    default(none) shared(a,n) \
    reduction(+:total)
for (int i = 0; i < n; ++i)
    total += a[i];
```

Private partial sums
+ a final combination
= one correct total

Trace the partial sums before combining



Illustrative assignment of [1, 2, 3, 4]

REFERENCE

Arithmetic reductions have identities

Identifier	Private initial value	Combination
+	0	Addition
*	1	Multiplication
min	Largest representable value	Minimum
max	Smallest representable value	Maximum
- (deprecated)	0	Addition

Initialize the original variable too; its value is part of the answer.

Bitwise and logical reductions differ

Identifier	Private initial value	Meaning
&	All bits set	Bitwise AND
|	0	Bitwise OR
^	0	Bitwise XOR
&&	1	All conditions true
||	0	At least one condition true

REFERENCE

Work the bit patterns by hand

Values	Three-bit form
1, 2, 3, 4	001, 010, 011, 100
AND	000 = 0
OR	111 = 7
XOR	100 = 4

Combine several results in one pass

```
#pragma omp parallel for \
  reduction(+:sum) reduction(*:product) \
  reduction(min:low) reduction(max:high)
for (int i = 0; i < 4; ++i) {
    sum += a[i];
    product *= a[i];
    if (a[i] < low) low = a[i];
    if (a[i] > high) high = a[i];
}
```

Input [1,2,3,4]

sum = 10
product = 24
min = 1
max = 4

Subtract contributions with a + reduction

```
int balance = 100;
#pragma omp parallel for \
    reduction(+:balance)
for (int i = 0; i < 4; ++i)
    balance -= a[i];

// 100 + (-3) + (-7) = 90
```

Private balances start at 0.

The original starts at 100.

Do not subtract partials from each other.

Initialization can change the answer

01 Product initialized to 0 stays 0.

02 Maximum initialized to 0 fails for all-negative data.

03 A private copy does not inherit the original value unless specified.

REFERENCE

Choose scope deliberately

Clause	Meaning	Typical use
shared(x)	One shared object	Input arrays; disjoint outputs
private(x)	Fresh uninitialized copy	Scratch assigned before use
firstprivate(x)	Copy the starting value	Stable task argument
reduction(+:x)	Private partials + combine	Accumulated result

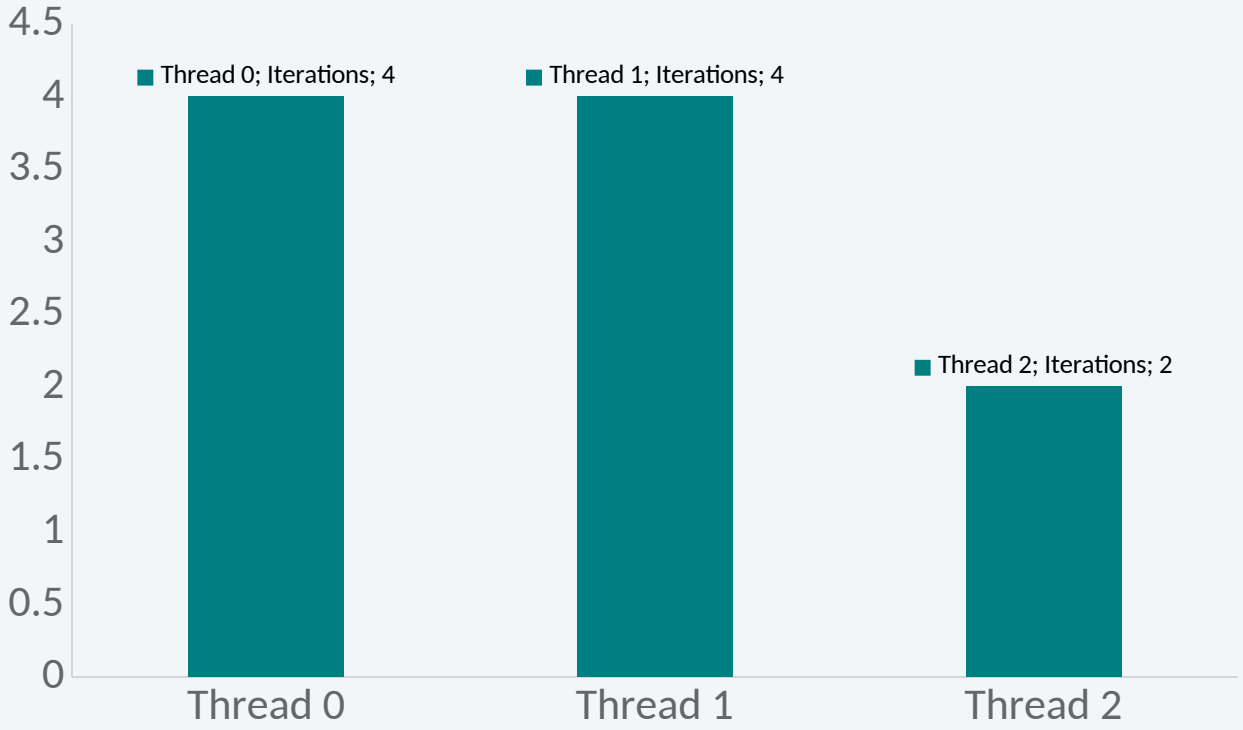
Scheduling starts after independence

01 First prove that iterations may execute in any order.

02 Then decide which thread receives each iteration.

03 Changing the schedule cannot repair a dependency.

Static chunks assign 4, 4, and 2 iterations



schedule(static,2)

T0: 0,1,6,7
T1: 2,3,8,9
T2: 4,5

Iteration counts, not elapsed times

REFERENCE

Match the schedule to the work

Schedule	Useful when	Tradeoff
static	Iteration costs are similar	Imbalance if costs cluster
dynamic,chunk	Costs are unpredictable	Assignment overhead
guided,chunk	Irregular work is substantial	Large early chunks may vary
runtime	Testing schedule choices	Uses OMP_SCHEDULE

Give one matrix row to each iteration

```
#pragma omp parallel for \
    default(none) shared(a,x,y)
for (int i = 0; i < ROWS; ++i) {
    int sum = 0;
    for (int j = 0; j < COLS; ++j)
        sum += a[i][j] * x[j];
    y[i] = sum;
}
```

One writer for y[i].

The local sum belongs to that iteration.

No cross-row reduction is needed.

Trace all four matrix rows

Row	Dot product with [1,2,3]	Output
[1,2,3]	1×1 + 2×2 + 3×3	14
[4,5,6]	4×1 + 5×2 + 6×3	32
[7,8,9]	7×1 + 8×2 + 9×3	50
[10,11,12]	10×1 + 11×2 + 12×3	68

collapse(2) exposes element-level work

```
#pragma omp parallel for collapse(2) \
    default(none) shared(a,doubled)
for (int i = 0; i < ROWS; ++i)
    for (int j = 0; j < COLS; ++j)
        doubled[i][j] = 2 * a[i][j];
```

4 rows × 3 columns = 12 independent updates

Each (i,j) owns one output.

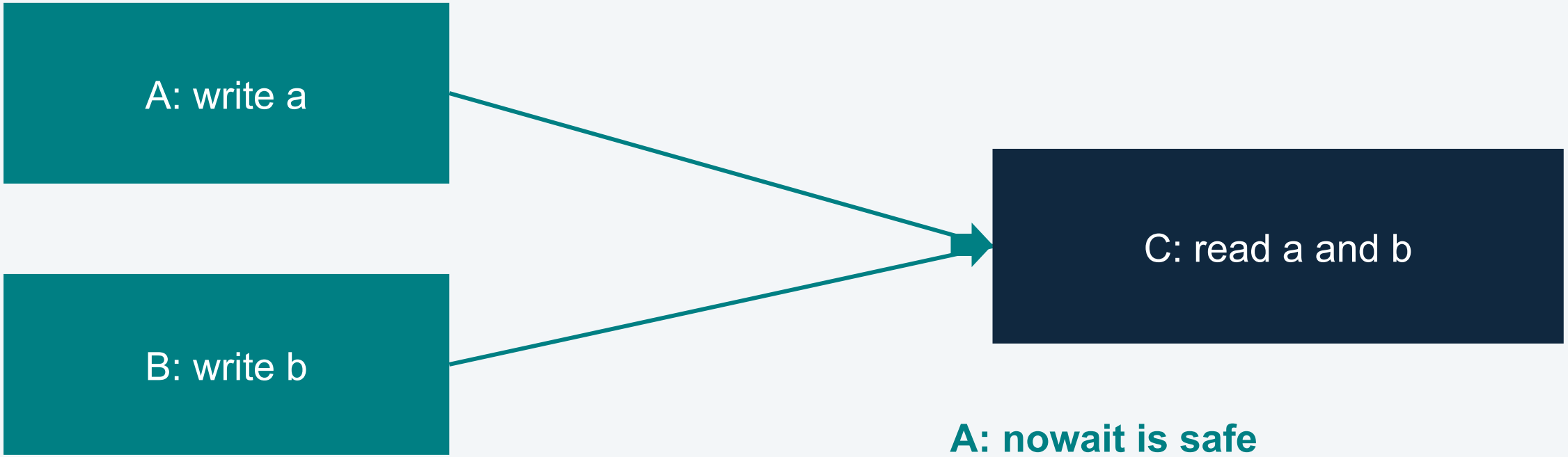
A barrier protects later reads

01 A worksharing for normally ends with a barrier.

02 nowait removes that barrier.

03 Remove it only when later work can safely proceed.

A and B may overlap; C needs both



A: nowait is safe
B: keep the barrier

Reuse one team across three loops

```
#pragma omp parallel shared(a,b,c)
{
  #pragma omp for nowait schedule(static)
  for (int i=0; i<N; ++i) a[i]=i+1;
  #pragma omp for schedule(static)
  for (int i=0; i<N; ++i) b[i]=2*(i+1);
  #pragma omp for schedule(static)
  for (int i=0; i<N; ++i)
    c[i]=a[i]+b[N-1-i];
}
```

A and B are independent.

B ends with a barrier.

C reads completed arrays.

Identify the read that needs the barrier

01 For N=4, C at i=0 reads b[3].

02 Another thread may still be computing b[3].

03 With the barrier: c[0] = 1 + 8 = 9.

REFERENCE

Choose synchronization by purpose

Construct	Purpose
reduction	Combine private contributions
atomic	Protect a supported update to one location
critical	Allow one thread at a time through a block
single	Execute a block once per team
barrier	Wait for all team threads

Tasks describe work for the existing team

01 A task can run immediately or later.

02 single creates the tasks once.

03 Any eligible team thread may execute a task.

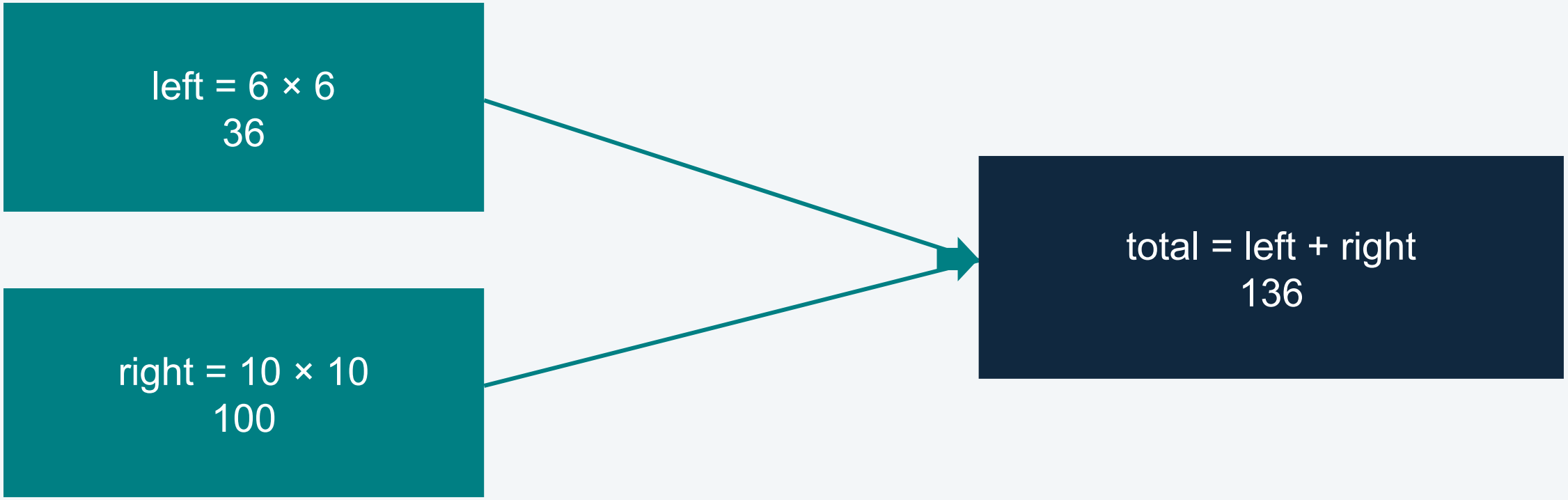
Capture a stable argument for each task

```
#pragma omp single
{
  for (int i=0; i<TASKS; ++i) {
    int slot = i;
    #pragma omp task firstprivate(slot) \
        shared(out)
    out[slot] = count_primes(5000*(slot+1));
  }
  #pragma omp taskwait
}
```

Task 0 captures slot=0.
Task 1 captures slot=1.

Each task owns a different out[slot].

A consumer task waits for two producers



Express producer–consumer dependencies

```
#pragma omp task shared(left) \
    depend(out:left)
left = 6 * 6;
#pragma omp task shared(right) \
    depend(out:right)
right = 10 * 10;
#pragma omp task shared(left,right,total) \
    depend(in:left,right) depend(out:total)
total = left + right;
```

Create these sibling tasks
inside single, within a team.

A taskgroup waits for the group to finish.

REFERENCE

Common errors have recognizable causes

Symptom / change	What to inspect
Serial execution	Compiler choice and -fopenmp
Wrong accumulated result	Missing reduction; incorrect initialization
Intermittent wrong output	Missing barrier or task dependency
Task writes wrong slot	Missing stable captured argument
Changing schedule does not help	Underlying loop dependency

Run the chapter’s checked examples

Source file	What to check
openmp_examples.c	sum=503506; scaled=1003; tasks=8
reduction_variants.c	sum=10; product=24; balance=90
matrix_vector.c	[14,32,50,68]; doubled=ok
worksharing_phases.c	verified=16 last=18
task_dependencies.c	left=36 right=100 total=136

Apply the ideas to three small changes

01 Count values greater than 500 using a reduction.

02 Explain why B cannot use nowait in the three-phase example.

03 Add a task that doubles total after the consumer.

Explain correctness before running faster

01 Assign one owner to each write, or combine with a reduction.

02 Keep data alive and make scope explicit.

03 Use barriers or task dependencies when later work needs earlier results.